KUL

USER GUIDE

ORFS Flow Runner

Install it, run a design from RTL to GDS and read the results, shown on real CVA6 runs, including four stock and parallel abc_new synthesis runs

AUTHOR	rafay
DATE	2026-10-26
APP VERSION	1.0.1
DESIGN AND PLATFORM	cva6 on asap7, hierarchical
MACHINE	Intel Core i5-1235U host, VirtualBox VM with 8 vCPUs and 10 GB RAM, Ubuntu 24.04

Contents

1. Summary 3

2. At a glance 3

3. Installing 4

- 3.1 Build and install the package 4
- 3.2 First start 4
- 3.3 Choose the tools 4

4. The window 6

5. Running a design 7

6. Reading the results 9

- 6.1 Reports → QoR checks 9
- 6.2 Reports → Report files 10
- 6.3 Logs 11
- 6.4 Debug 12

7. Worked example: stock and parallel abc_new 13

- 7.1 What was compared 13
- 7.2 How to repeat it 14
- 7.3 Results 14
- 7.4 What the numbers mean 15

8. Other tasks 16

- 8.1 Re-run from a stage 16
- 8.2 Runs that must outlive the app 17
- 8.3 Open a stage in the OpenROAD GUI 17

9. Troubleshooting 17

10. Files 18

1. Summary

ORFS Flow Runner is a desktop app for OpenROAD-flow-scripts (ORFS). It runs any design in an ORFS flow folder from RTL to GDS through the stock ORFS Makefile, so a run in the app gives the same result as the same make command in a terminal. This guide shows every part of the app with screenshots taken from real runs on this machine.

- **Install:** one Debian package, `orfs-flow-runner_1.0.1_all.deb`, puts the app under /opt with a menu entry. OpenROAD, Yosys and ORFS are installed separately; the app finds them on PATH or where Tools… points.
- **Run:** pick platform, design, config, CPU cores and a stage range from drop-down menus. The plan box shows the exact make command before anything runs.
- **Read results:** Reports shows pass/fail tiles and a metric-by-stage table; Logs and Report files have a line filter; Debug shows the environment, the resolved make variables and every warning ID.
- **Worked example:** CVA6 (asap7, hierarchical) synthesised four times from the app: stock Yosys, parallel abc_new, parallel abc_new, stock Yosys. The netlists are identical in all four. abc_new was 1.29× faster in the cleanest pair of runs, but the machine slowed down during the later runs, and in that pair the parallel abc_new was slower (0.85×). Averaged over two runs per build, abc_new is 1.02× and the whole synthesis 1.12× faster. Run-to-run noise on this machine is as large as the effect, so section 7 reports the range, not one number.

2. At a glance

Item	Value
App version	1.0.1 (gui/orfs_app.py)
Package	orfs-flow-runner_1.0.1_all.deb, built by gui/packaging/build_deb.sh
Needs	Python 3, GTK 3 with PyGObject, make; KLayout for the final GDS step
Installed separately	OpenROAD, Yosys, an OpenROAD-flow-scripts checkout
Tested on	Ubuntu 24.04 in a VirtualBox VM: Intel Core i5-1235U host, 8 vCPUs, 10 GB RAM
Tool versions in the screenshots	OpenROAD 26Q4-271-g0907a8656e; Yosys 0.69+274 (stock) and 0.69+280 (parallel abc_new); KLayout 0.28.16
Design in the screenshots	cva6 on asap7, hierarchical synthesis (config_hier*.mk)
Per-user files	~/.config/orfs-flow-runner, ~/.local/share/orfs-flow-runner/run_logs, ~/.cache/orfs-flow-runner

3. Installing

3.1 Build and install the package

From a checkout of KUL, build the package and install it with apt. apt also installs GTK, PyGObject, make and KLayout when they are missing.

```
gui/packaging/build_deb.sh
sudo apt install ./dist/orfs-flow-runner_1.0.1_all.deb
```

The package installs the app in /opt/orfs-flow-runner, adds ORFS Flow Runner to the app menu and adds an orfs-flow-runner command. To update, build and install the newer package the same way. sudo apt remove orfs-flow-runner removes it and keeps your settings and run logs.

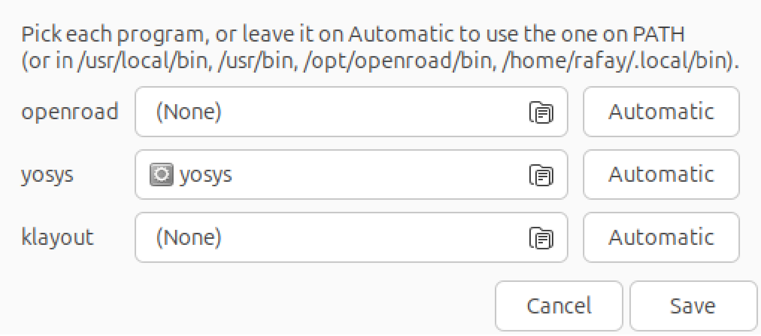
If you used gui/install_launcher.sh before, run gui/install_launcher.sh --uninstall first: its menu entry has the same name and would hide the installed one.

3.2 First start

Start ORFS Flow Runner from the app menu. It opens the flow folder you used last, else $ORFS_FLOW, else ~/OpenROAD-flow-scripts/flow. If it finds none it asks for one; pick the flow/ folder of your ORFS checkout. Flow folder... in the title bar changes it later.

3.3 Choose the tools

Tools... in the title bar sets which OpenROAD, Yosys and KLayout programs the runs use. Leave a row on Automatic to use the program on PATH, or the one in /usr/local/bin, /usr/bin, /opt/openroad/bin or ~/.local/bin. An app started from the menu has a short PATH, so this is how it finds tools installed elsewhere. Choices are saved in ~/.config/orfs-flow-runner/settings.json.

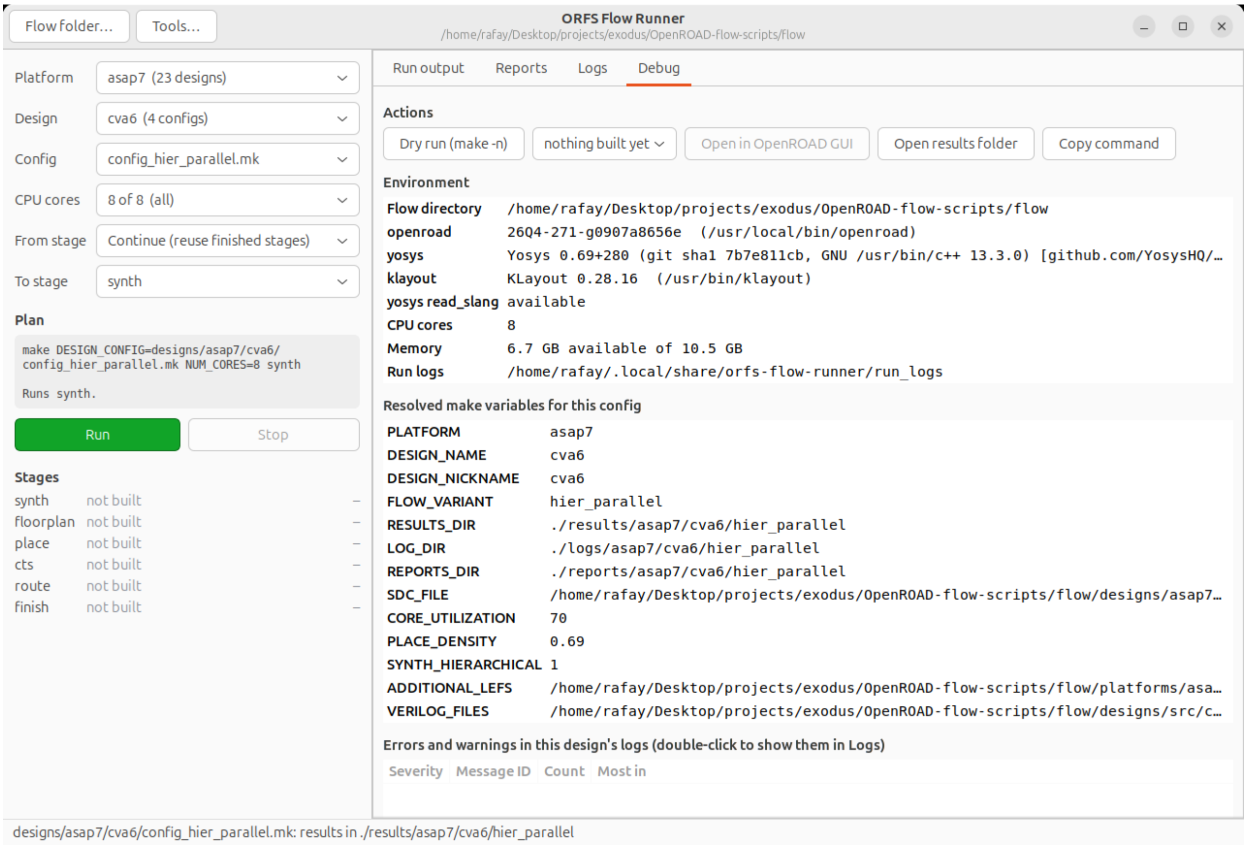


Pick each program, or leave it on Automatic to use the one on PATH
(or in /usr/local/bin, /usr/bin, /opt/openroad/bin, /home/rafay/.local/bin).

openroad: (None)
yosys: yosys
klayout: (None)

Figure 1. Tools... with Yosys set to the parallel abc_new build (yosys/build/yosys); OpenROAD and KLayout stay on Automatic.

Debug → Environment then shows the version and path of each tool, whether Yosys has the read_slang frontend that CVA6 needs, the CPU count, free memory and the run-log folder. Check it once after installing: a NOT FOUND row means a run would fail at once.


Flow folder…
Tools…
ORFS Flow Runner
/home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow
Run output
Reports
Logs
Debug
Run
Stop
Dry run (make -n)
nothing built yet
Open in OpenROAD GUI
Open results folder
Copy command

Platform	asap7 (23 designs)
Design	cva6 (4 configs)
Config	config_hier_parallel.mk
CPU cores	8 of 8 (all)
From stage	Continue (reuse finished stages)
To stage	synth

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier_parallel.mk NUM_CORES=8 synth

Runs synth.
```

Stages

synth	not built	–
floorplan	not built	–
place	not built	–
cts	not built	–
route	not built	–
finish	not built	–

Actions

Environment

Flow directory	`/home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow`
openroad	`26Q4-271-g0907a8656e  (/usr/local/bin/openroad)`
yosys	`Yosys 0.69+280 (git sha1 7b7e811cb, GNU /usr/bin/c++ 13.3.0) [github.com/YosysHQ/…`
klayout	`KLayout 0.28.16  (/usr/bin/klayout)`
yosys read_slang	`available`
CPU cores	`8`
Memory	`6.7 GB available of 10.5 GB`
Run logs	`/home/rafay/.local/share/orfs-flow-runner/run_logs`

Resolved make variables for this config

PLATFORM	`asap7`
DESIGN_NAME	`cva6`
DESIGN_NICKNAME	`cva6`
FLOW_VARIANT	`hier_parallel`
RESULTS_DIR	`./results/asap7/cva6/hier_parallel`
LOG_DIR	`./logs/asap7/cva6/hier_parallel`
REPORTS_DIR	`./reports/asap7/cva6/hier_parallel`
SDC_FILE	`/home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/designs/asap7…`
CORE_UTILIZATION	`70`
PLACE_DENSITY	`0.69`
SYNTH_HIERARCHICAL	`1`
ADDITIONAL_LEFS	`/home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms/asa…`
VERILOG_FILES	`/home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/designs/src/c…`

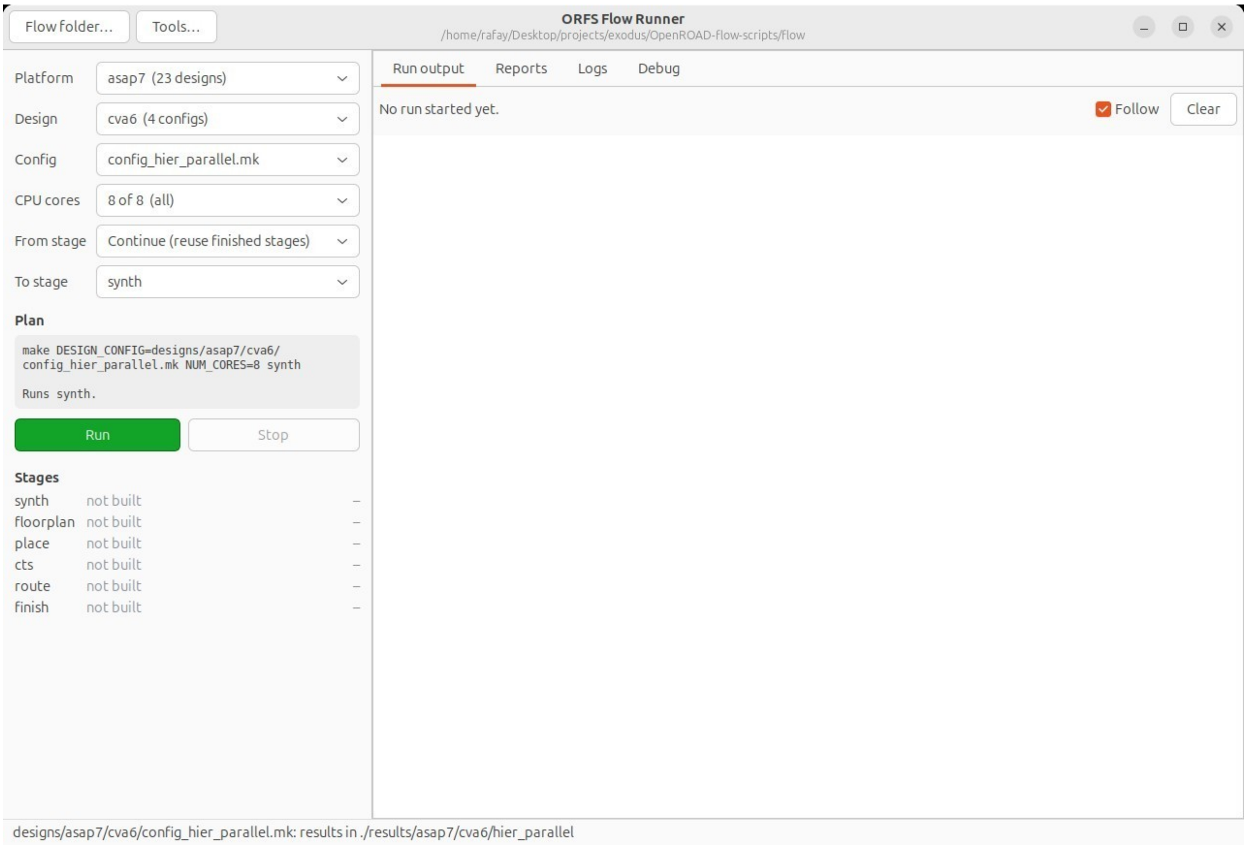
Errors and warnings in this design's logs (double-click to show them in Logs)

Severity	Message ID	Count	Most in

designs/asap7/cva6/config_hier_parallel.mk: results in ./results/asap7/cva6/hier_parallel

Figure 2. Debug → Environment after choosing the parallel Yosys: openroad, yosys and klayout found, read_slang available, logs in the per-user folder.

4. The window



Platform: asap7 (23 designs)
Design: cva6 (4 configs)
Config: config_hier_parallel.mk
CPU cores: 8 of 8 (all)
From stage: Continue (reuse finished stages)
To stage: synth

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier_parallel.mk NUM_CORES=8 synth

Runs synth.
```

Stages

synth	not built	–
floorplan	not built	–
place	not built	–
cts	not built	–
route	not built	–
finish	not built	–

No run started yet.

designs/asap7/cva6/config_hier_parallel.mk: results in ./results/asap7/cva6/hier_parallel

Figure 3. The window before a run: menus and plan on the left, output and results tabs on the right, status bar at the bottom.

Part	What it does
Title bar	Flow folder... and Tools...; the subtitle is the flow folder in use
Platform	every folder in flow/designs with designs; platforms whose PDK is missing are greyed out
Design, Config	the designs of that platform and their config*.mk files; the config sets DESIGN_CONFIG and with it the results folder
CPU cores	passed as NUM_CORES; fewer cores use less memory
From stage	Continue (reuse finished stages) or Re-run from a stage, which cleans that stage and every later one first
To stage	the last make target: synth, floorplan, place, cts, route or finish
Plan	the exact commands, the stages they run and anything they delete
Run, Stop	start the plan; Stop kills the running step and keeps finished stages
Stages	each stage as not built, running, partial, interrupted or done, with its runtime and peak memory
Tabs	Run output, Reports, Logs, Debug (sections 5 and 6)

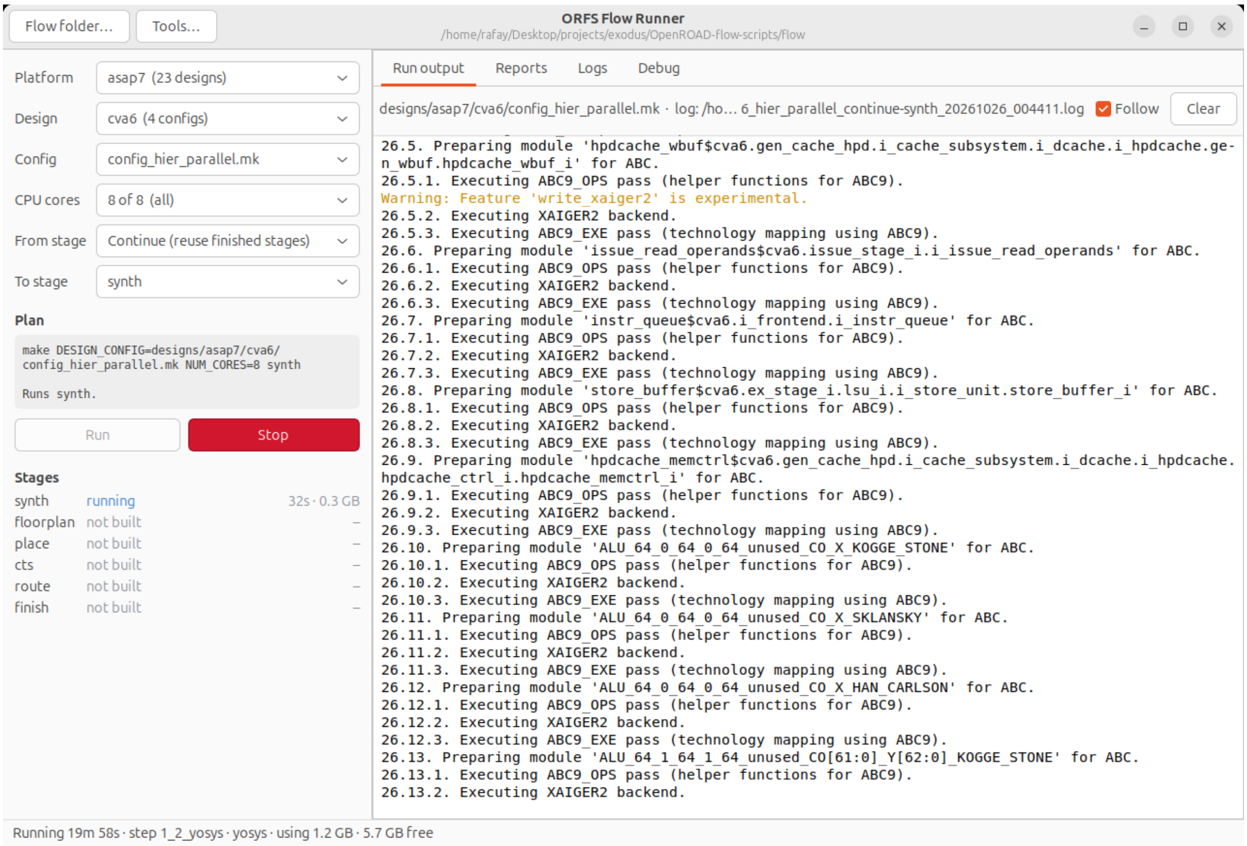
Part	What it does
Status bar	while running: elapsed time, current step, tool, memory used and free; after: the result and the log path

The app remembers the flow folder, the menu choices and the tools for the next start.

5. Running a design

This walk-through synthesises CVA6 with hierarchical synthesis. Smaller designs such as nangate45/gcd work the same way and finish in minutes.

1. **Platform:** asap7. **Design:** cva6. **Config:** config_hier_parallel.mk (section 7 explains the A/B configs; config_hier.mk is the plain one).
2. **CPU cores:** 8 of 8. **From stage:** Continue. **To stage:** synth.
3. **Read the plan box.** It reads make DESIGN_CONFIG=designs/asap7/cva6/config_hier_parallel.mk NUM_CORES=8 synth and Runs synth.
4. **Press Run.** Run output streams the output: errors in red, warnings in amber, the run's own messages in blue. Untick Follow to scroll back without being pulled to the end.
5. **Watch Stages and the status bar.** synth shows running, and the status bar names the step (1_2_yosys), the tool and the memory in use.
6. **Wait for >> exit code 0.** The status bar then says Run finished after the elapsed time, with the path of the run log.



Platform: asap7 (23 designs)
Design: cva6 (4 configs)
Config: config_hier_parallel.mk
CPU cores: 8 of 8 (all)
From stage: Continue (reuse finished stages)
To stage: synth

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier_parallel.mk NUM_CORES=8 synth

Runs synth.
```

Stages

synth	running	32s · 0.3 GB
floorplan	not built	–
place	not built	–
cts	not built	–
route	not built	–
finish	not built	–

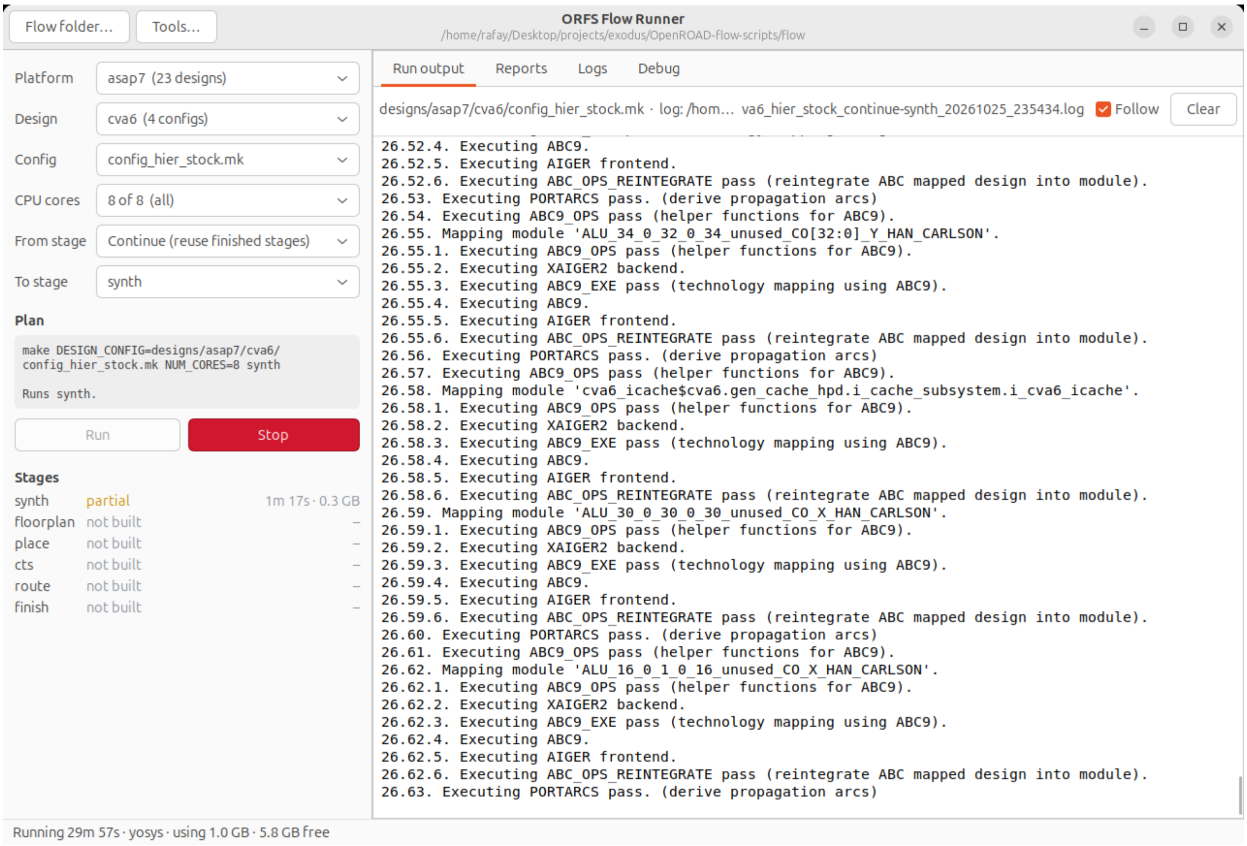
designs/asap7/cva6/config_hier_parallel.mk · log: /ho… 6_hier_parallel_continue-synth_20261026_004411.log ☑ Follow

```
26.5. Preparing module 'hpdcache_wbuf$cva6.gen_cache_hpd.i_cache_subsystem.i_dcache.i_hpdcache.ge-
n_wbuf.hpdcache_wbuf_i' for ABC.
26.5.1. Executing ABC9_OPS pass (helper functions for ABC9).
Warning: Feature 'write_xaiger2' is experimental.
26.5.2. Executing XAIGER2 backend.
26.5.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.6. Preparing module 'issue_read_operands$cva6.issue_stage_i.i_issue_read_operands' for ABC.
26.6.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.6.2. Executing XAIGER2 backend.
26.6.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.7. Preparing module 'instr_queue$cva6.i_frontend.i_instr_queue' for ABC.
26.7.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.7.2. Executing XAIGER2 backend.
26.7.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.8. Preparing module 'store_buffer$cva6.ex_stage_i.lsu_i.i_store_unit.store_buffer_i' for ABC.
26.8.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.8.2. Executing XAIGER2 backend.
26.8.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.9. Preparing module 'hpdcache_memctrl$cva6.gen_cache_hpd.i_cache_subsystem.i_dcache.i_hpdcache.
hpdcache_ctrl_i.hpdcache_memctrl_i' for ABC.
26.9.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.9.2. Executing XAIGER2 backend.
26.9.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.10. Preparing module 'ALU_64_0_64_0_64_unused_CO_X_KOGGE_STONE' for ABC.
26.10.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.10.2. Executing XAIGER2 backend.
26.10.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.11. Preparing module 'ALU_64_0_64_0_64_unused_CO_X_SKLANSKY' for ABC.
26.11.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.11.2. Executing XAIGER2 backend.
26.11.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.12. Preparing module 'ALU_64_0_64_0_64_unused_CO_X_HAN_CARLSON' for ABC.
26.12.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.12.2. Executing XAIGER2 backend.
26.12.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.13. Preparing module 'ALU_64_1_64_1_64_unused_CO[61:0]_Y[62:0]_KOGGE_STONE' for ABC.
26.13.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.13.2. Executing XAIGER2 backend.
```

Running 19m 58s · step 1_2_yosys · yosys · using 1.2 GB · 5.7 GB free

Figure 4. Twenty minutes into synthesis with the parallel Yosys: the step is 1_2_yosys and abc_new is preparing each module for ABC.

With stock Yosys the same point of the run looks different: abc_new maps one module at a time, so the output alternates Mapping module and ABC9 lines for each of the 93 modules (Figure 5).



Platform: asap7 (23 designs)
Design: cva6 (4 configs)
Config: config_hier_stock.mk
CPU cores: 8 of 8 (all)
From stage: Continue (reuse finished stages)
To stage: synth

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier_stock.mk NUM_CORES=8 synth

Runs synth.
```

Stages

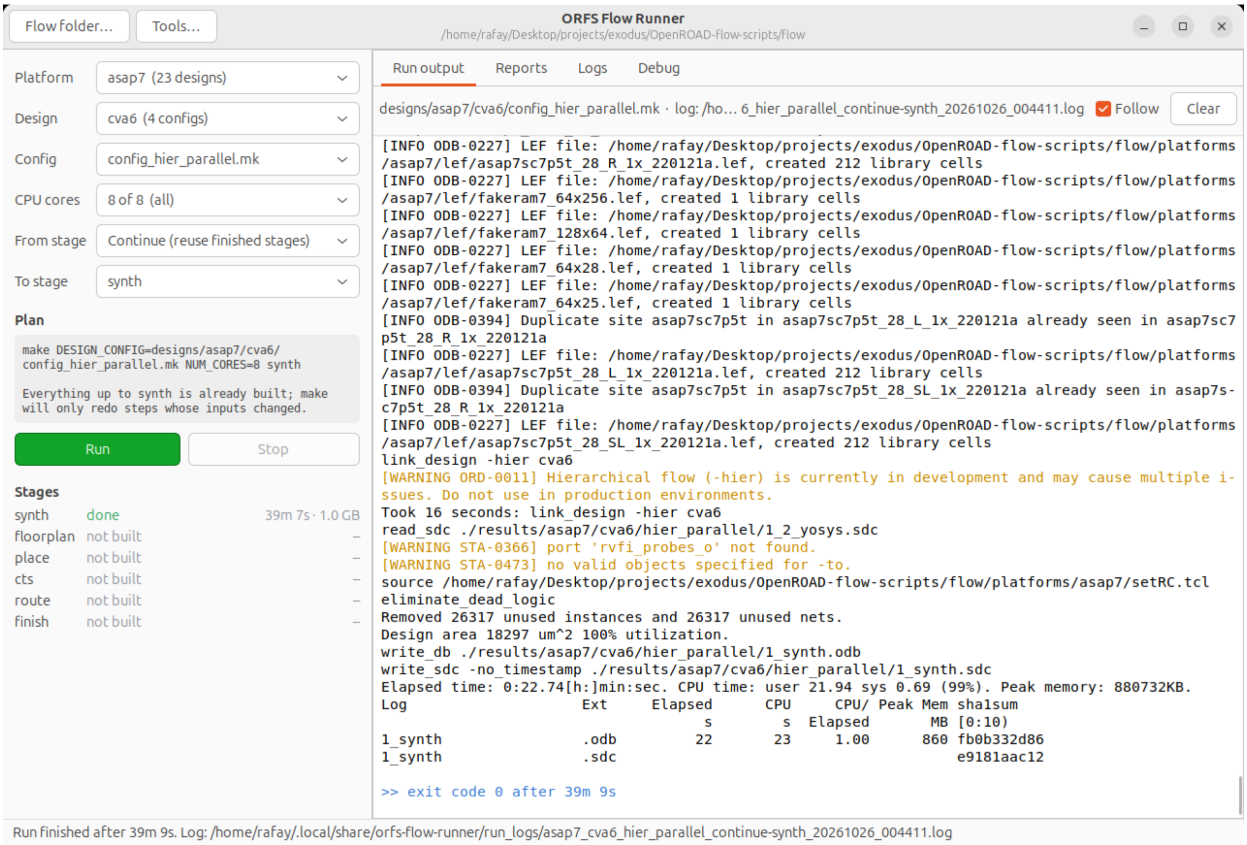
synth	partial	1m 17s · 0.3 GB
floorplan	not built	–
place	not built	–
cts	not built	–
route	not built	–
finish	not built	–

designs/asap7/cva6/config_hier_stock.mk · log: /hom… va6_hier_stock_continue-synth_20261025_235434.log

```
26.52.4. Executing ABC9.
26.52.5. Executing AIGER frontend.
26.52.6. Executing ABC_OPS_REINTEGRATE pass (reintegrate ABC mapped design into module).
26.53. Executing PORTARCS pass. (derive propagation arcs)
26.54. Executing ABC9_OPS pass (helper functions for ABC9).
26.55. Mapping module 'ALU_34_0_32_0_34_unused_CO[32:0]_Y_HAN_CARLSON'.
26.55.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.55.2. Executing XAIGER2 backend.
26.55.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.55.4. Executing ABC9.
26.55.5. Executing AIGER frontend.
26.55.6. Executing ABC_OPS_REINTEGRATE pass (reintegrate ABC mapped design into module).
26.56. Executing PORTARCS pass. (derive propagation arcs)
26.57. Executing ABC9_OPS pass (helper functions for ABC9).
26.58. Mapping module 'cva6_icache$cva6.gen_cache_hpd.i_cache_subsystem.i_cva6_icache'.
26.58.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.58.2. Executing XAIGER2 backend.
26.58.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.58.4. Executing ABC9.
26.58.5. Executing AIGER frontend.
26.58.6. Executing ABC_OPS_REINTEGRATE pass (reintegrate ABC mapped design into module).
26.59. Mapping module 'ALU_30_0_30_0_30_unused_CO_X_HAN_CARLSON'.
26.59.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.59.2. Executing XAIGER2 backend.
26.59.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.59.4. Executing ABC9.
26.59.5. Executing AIGER frontend.
26.59.6. Executing ABC_OPS_REINTEGRATE pass (reintegrate ABC mapped design into module).
26.60. Executing PORTARCS pass. (derive propagation arcs)
26.61. Executing ABC9_OPS pass (helper functions for ABC9).
26.62. Mapping module 'ALU_16_0_1_0_16_unused_CO_X_HAN_CARLSON'.
26.62.1. Executing ABC9_OPS pass (helper functions for ABC9).
26.62.2. Executing XAIGER2 backend.
26.62.3. Executing ABC9_EXE pass (technology mapping using ABC9).
26.62.4. Executing ABC9.
26.62.5. Executing AIGER frontend.
26.62.6. Executing ABC_OPS_REINTEGRATE pass (reintegrate ABC mapped design into module).
26.63. Executing PORTARCS pass. (derive propagation arcs)
```

Running 29m 57s · yosys · using 1.0 GB · 5.8 GB free

Figure 5. The same step with stock Yosys, 30 minutes in. Screenshot from app version 1.0.0, which showed synth as partial while it ran; 1.0.1 shows running.



Platform: asap7 (23 designs)
Design: cva6 (4 configs)
Config: config_hier_parallel.mk
CPU cores: 8 of 8 (all)
From stage: Continue (reuse finished stages)
To stage: synth

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier_parallel.mk NUM_CORES=8 synth

Everything up to synth is already built; make
will only redo steps whose inputs changed.
```

Stages

synth	done	39m 7s · 1.0 GB
floorplan	not built	–
place	not built	–
cts	not built	–
route	not built	–
finish	not built	–

designs/asap7/cva6/config_hier_parallel.mk · log: /ho… 6_hier_parallel_continue-synth_20261026_004411.log

```
[INFO ODB-0227] LEF file: /home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms
/asap7/lef/asap7sc7p5t_28_R_1x_220121a.lef, created 212 library cells
[INFO ODB-0227] LEF file: /home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms
/asap7/lef/fakeram7_64x256.lef, created 1 library cells
[INFO ODB-0227] LEF file: /home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms
/asap7/lef/fakeram7_128x64.lef, created 1 library cells
[INFO ODB-0227] LEF file: /home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms
/asap7/lef/fakeram7_64x28.lef, created 1 library cells
[INFO ODB-0227] LEF file: /home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms
/asap7/lef/fakeram7_64x25.lef, created 1 library cells
[INFO ODB-0394] Duplicate site asap7sc7p5t in asap7sc7p5t_28_L_1x_220121a already seen in asap7sc7
p5t_28_R_1x_220121a
[INFO ODB-0227] LEF file: /home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms
/asap7/lef/asap7sc7p5t_28_L_1x_220121a.lef, created 212 library cells
[INFO ODB-0394] Duplicate site asap7sc7p5t in asap7sc7p5t_28_SL_1x_220121a already seen in asap7s-
c7p5t_28_R_1x_220121a
[INFO ODB-0227] LEF file: /home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms
/asap7/lef/asap7sc7p5t_28_SL_1x_220121a.lef, created 212 library cells
link_design -hier cva6
[WARNING ORD-0011] Hierarchical flow (-hier) is currently in development and may cause multiple i-
ssues. Do not use in production environments.
Took 16 seconds: link_design -hier cva6
read_sdc ./results/asap7/cva6/hier_parallel/1_2_yosys.sdc
[WARNING STA-0366] port 'rvfi_probes_o' not found.
[WARNING STA-0473] no valid objects specified for -to.
source /home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms/asap7/setRC.tcl
eliminate_dead_logic
Removed 26317 unused instances and 26317 unused nets.
Design area 18297 um^2 100% utilization.
write_db ./results/asap7/cva6/hier_parallel/1_synth.odb
write_sdc -no_timestamp ./results/asap7/cva6/hier_parallel/1_synth.sdc
Elapsed time: 0:22.74[h:]min:sec. CPU time: user 21.94 sys 0.69 (99%). Peak memory: 880732KB.
Log                    Ext     Elapsed      CPU     CPU/ Peak Mem sha1sum
                                     s        s  Elapsed       MB [0:10)
1_synth                .odb         22       23     1.00      860 fb0b332d86
1_synth                .sdc                                       e9181aac12

>> exit code 0 after 39m 9s
```

Run finished after 39m 9s. Log: /home/rafay/.local/share/orfs-flow-runner/run_logs/asap7_cva6_hier_parallel_continue-synth_20261026_004411.log

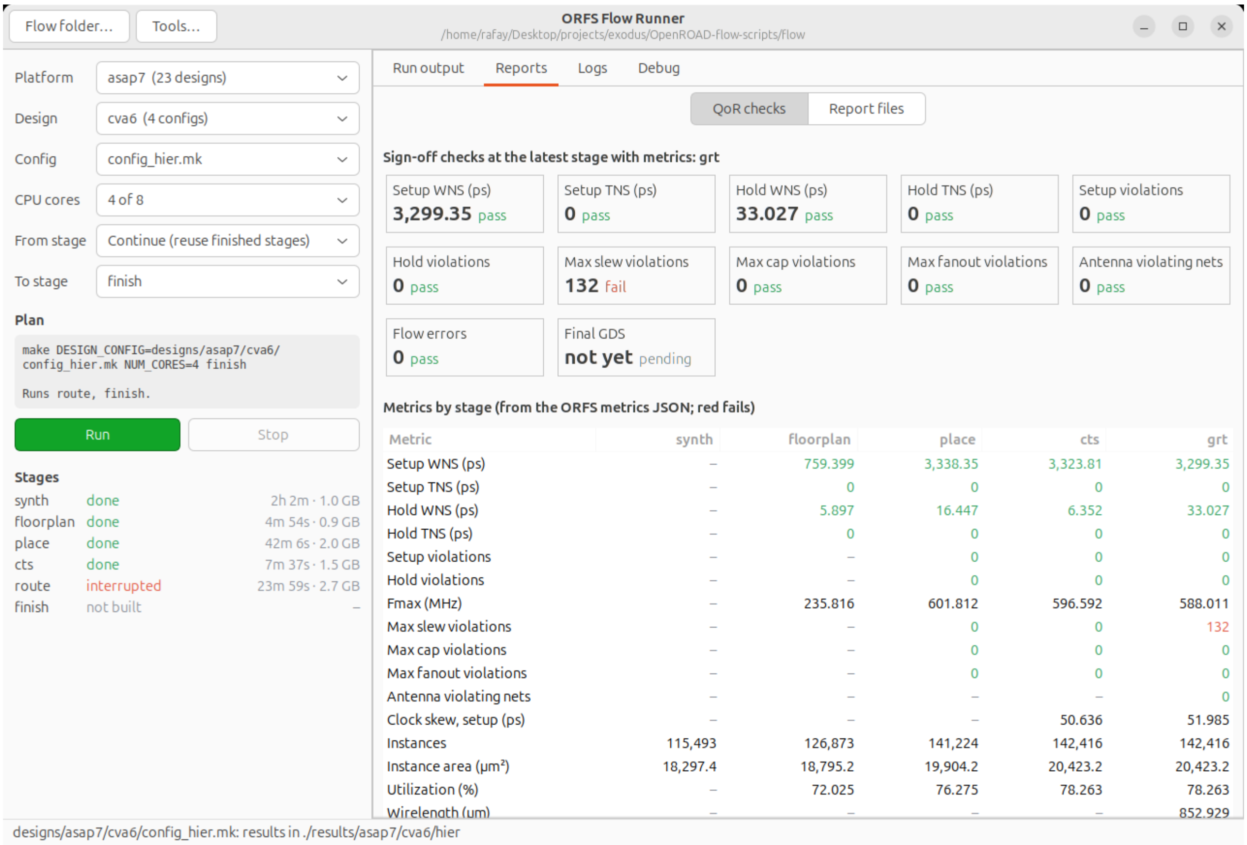
Figure 6. A finished synthesis run: synth done with its runtime and 1.0 GB peak, exit code 0, and the run log path in the status bar. From the first single-pair attempt; its timings are superseded by section 7.

To stop a run, press Stop and confirm. The running step is killed, its partial log stays as a .tmp.log and finished stages are kept; the stage then shows interrupted. Closing the window during a run asks first.

6. Reading the results

6.1 Reports → QoR checks

QoR checks has pass/fail tiles for the latest stage that has metrics, then a table of every metric by stage. Red marks a failure: negative slack or a non-zero violation count. The numbers come from the ORFS metrics JSON files in logs/<platform>/<design>/<variant>/.



Platform: asap7 (23 designs)
Design: cva6 (4 configs)
Config: config_hier.mk
CPU cores: 4 of 8
From stage: Continue (reuse finished stages)
To stage: finish

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier.mk NUM_CORES=4 finish

Runs route, finish.
```

Stages

synth	done	2h 2m · 1.0 GB
floorplan	done	4m 54s · 0.9 GB
place	done	42m 6s · 2.0 GB
cts	done	7m 37s · 1.5 GB
route	interrupted	23m 59s · 2.7 GB
finish	not built	–

Sign-off checks at the latest stage with metrics: grt

Setup WNS (ps) **3,299.35** pass	Setup TNS (ps) **0** pass	Hold WNS (ps) **33.027** pass	Hold TNS (ps) **0** pass	Setup violations **0** pass
Hold violations **0** pass	Max slew violations **132** fail	Max cap violations **0** pass	Max fanout violations **0** pass	Antenna violating nets **0** pass
Flow errors **0** pass	Final GDS **not yet** pending			

Metrics by stage (from the ORFS metrics JSON; red fails)

Metric	synth	floorplan	place	cts	grt
Setup WNS (ps)	–	759.399	3,338.35	3,323.81	3,299.35
Setup TNS (ps)	–	0	0	0	0
Hold WNS (ps)	–	5.897	16.447	6.352	33.027
Hold TNS (ps)	–	0	0	0	0
Setup violations	–	–	0	0	0
Hold violations	–	–	0	0	0
Fmax (MHz)	–	235.816	601.812	596.592	588.011
Max slew violations	–	–	0	0	132
Max cap violations	–	–	0	0	0
Max fanout violations	–	–	0	0	0
Antenna violating nets	–	–	–	–	0
Clock skew, setup (ps)	–	–	–	50.636	51.985
Instances	115,493	126,873	141,224	142,416	142,416
Instance area (µm²)	18,297.4	18,795.2	19,904.2	20,423.2	20,423.2
Utilization (%)	–	72.025	76.275	78.263	78.263
Wirelength (µm)	–	–	–	–	852.929

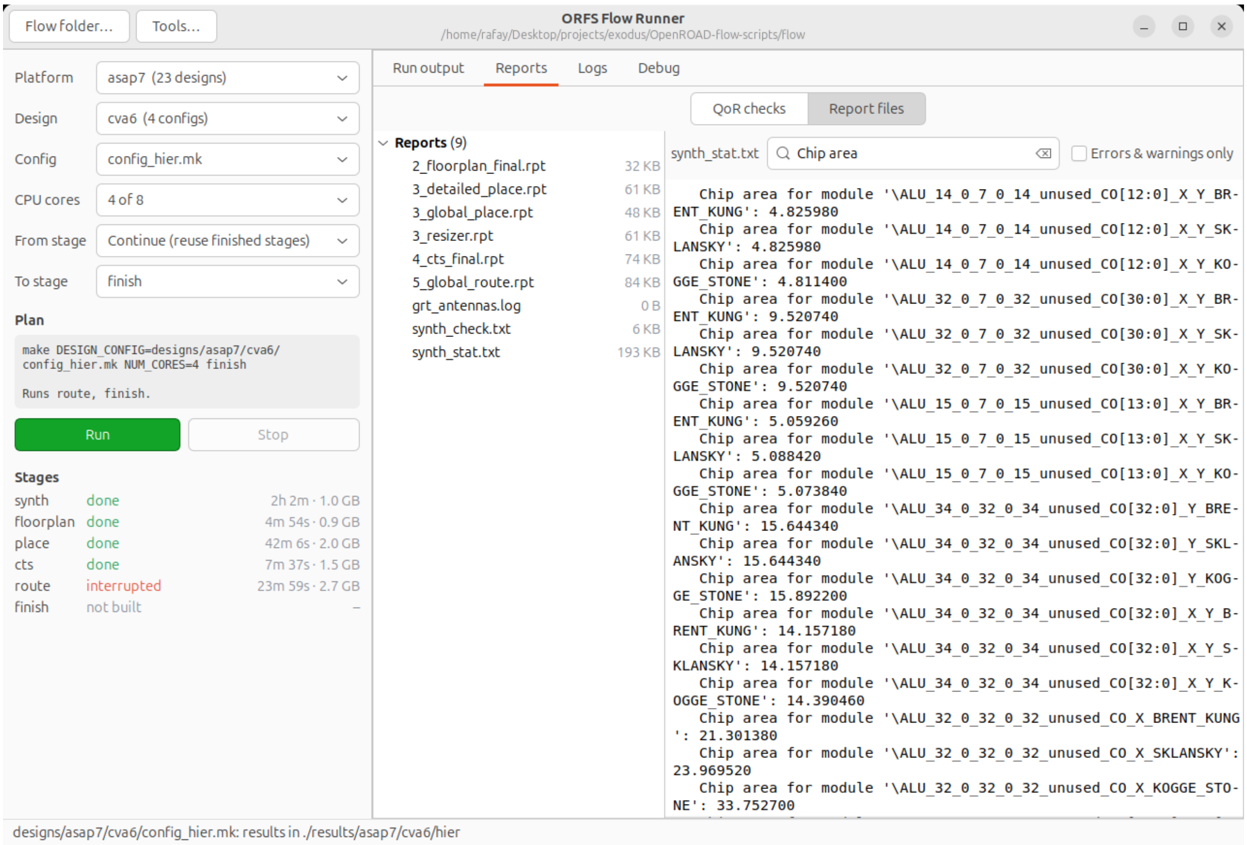
designs/asap7/cva6/config_hier.mk: results in ./results/asap7/cva6/hier

Figure 7. QoR checks for the earlier CVA6 hier run, which reached global routing before detailed routing was interrupted: timing passes, 132 max-slew violations fail.

After synthesis only, the table has one column: 115,493 instances and 18,297.4 µm^2 for CVA6, with 0 flow errors and 33 warnings. All four A/B runs show exactly these numbers.

6.2 Reports → Report files

Report files lists the design's reports folder. Pick a file to read it; type in the filter box to keep only matching lines, or tick Errors & warnings only.



Platform: asap7 (23 designs)
Design: cva6 (4 configs)
Config: config_hier.mk
CPU cores: 4 of 8
From stage: Continue (reuse finished stages)
To stage: finish

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier.mk NUM_CORES=4 finish

Runs route, finish.
```

Stages

synth	done	2h 2m · 1.0 GB
floorplan	done	4m 54s · 0.9 GB
place	done	42m 6s · 2.0 GB
cts	done	7m 37s · 1.5 GB
route	interrupted	23m 59s · 2.7 GB
finish	not built	–

Reports (9)

2_floorplan_final.rpt	32 KB
3_detailed_place.rpt	61 KB
3_global_place.rpt	48 KB
3_resizer.rpt	61 KB
4_cts_final.rpt	74 KB
5_global_route.rpt	84 KB
grt_antennas.log	0 B
synth_check.txt	6 KB
synth_stat.txt	193 KB

synth_stat.txt Chip area ☐ Errors & warnings only

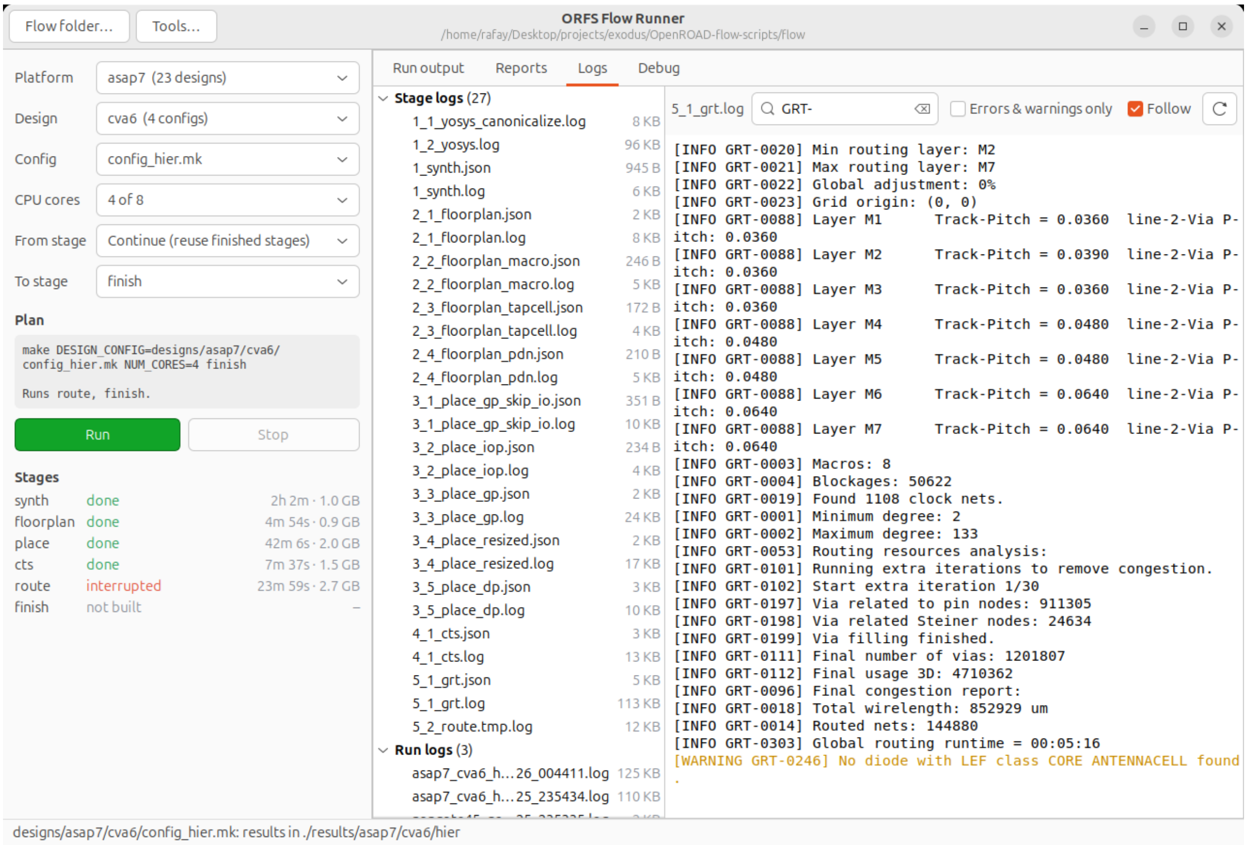
```
   Chip area for module '\ALU_14_0_7_0_14_unused_CO[12:0]_X_Y_BR-
ENT_KUNG': 4.825980
   Chip area for module '\ALU_14_0_7_0_14_unused_CO[12:0]_X_Y_SK-
LANSKY': 4.825980
   Chip area for module '\ALU_14_0_7_0_14_unused_CO[12:0]_X_Y_KO-
GGE_STONE': 4.811400
   Chip area for module '\ALU_32_0_7_0_32_unused_CO[30:0]_X_Y_BR-
ENT_KUNG': 9.520740
   Chip area for module '\ALU_32_0_7_0_32_unused_CO[30:0]_X_Y_SK-
LANSKY': 9.520740
   Chip area for module '\ALU_32_0_7_0_32_unused_CO[30:0]_X_Y_KO-
GGE_STONE': 9.520740
   Chip area for module '\ALU_15_0_7_0_15_unused_CO[13:0]_X_Y_BR-
ENT_KUNG': 5.059260
   Chip area for module '\ALU_15_0_7_0_15_unused_CO[13:0]_X_Y_SK-
LANSKY': 5.088420
   Chip area for module '\ALU_15_0_7_0_15_unused_CO[13:0]_X_Y_KO-
GGE_STONE': 5.073840
   Chip area for module '\ALU_34_0_32_0_34_unused_CO[32:0]_Y_BRE-
NT_KUNG': 15.644340
   Chip area for module '\ALU_34_0_32_0_34_unused_CO[32:0]_Y_SKL-
ANSKY': 15.644340
   Chip area for module '\ALU_34_0_32_0_34_unused_CO[32:0]_Y_KOG-
GE_STONE': 15.892200
   Chip area for module '\ALU_34_0_32_0_34_unused_CO[32:0]_X_Y_B-
RENT_KUNG': 14.157180
   Chip area for module '\ALU_34_0_32_0_34_unused_CO[32:0]_X_Y_S-
KLANSKY': 14.157180
   Chip area for module '\ALU_34_0_32_0_34_unused_CO[32:0]_X_Y_K-
OGGE_STONE': 14.390460
   Chip area for module '\ALU_32_0_32_0_32_unused_CO_X_BRENT_KUNG
': 21.301380
   Chip area for module '\ALU_32_0_32_0_32_unused_CO_X_SKLANSKY':
23.969520
   Chip area for module '\ALU_32_0_32_0_32_unused_CO_X_KOGGE_STO-
NE': 33.752700
```

designs/asap7/cva6/config_hier.mk: results in ./results/asap7/cva6/hier

Figure 8. synth_stat.txt filtered to Chip area: the area of each synthesised module.

6.3 Logs

Logs lists the stage logs (one per step; a .tmp.log is a step that is running or was killed) and the run logs (one per run, newest first). The filter and Errors & warnings only work as in Report files. With Follow ticked, an open log that is still being written keeps scrolling to its end.



Platform: asap7 (23 designs)
Design: cva6 (4 configs)
Config: config_hier.mk
CPU cores: 4 of 8
From stage: Continue (reuse finished stages)
To stage: finish

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier.mk NUM_CORES=4 finish

Runs route, finish.
```

Stages

synth	done	2h 2m · 1.0 GB
floorplan	done	4m 54s · 0.9 GB
place	done	42m 6s · 2.0 GB
cts	done	7m 37s · 1.5 GB
route	interrupted	23m 59s · 2.7 GB
finish	not built	–

Stage logs (27)

1_1_yosys_canonicalize.log	8 KB
1_2_yosys.log	96 KB
1_synth.json	945 B
1_synth.log	6 KB
2_1_floorplan.json	2 KB
2_1_floorplan.log	8 KB
2_2_floorplan_macro.json	246 B
2_2_floorplan_macro.log	5 KB
2_3_floorplan_tapcell.json	172 B
2_3_floorplan_tapcell.log	4 KB
2_4_floorplan_pdn.json	210 B
2_4_floorplan_pdn.log	5 KB
3_1_place_gp_skip_io.json	351 B
3_1_place_gp_skip_io.log	10 KB
3_2_place_iop.json	234 B
3_2_place_iop.log	4 KB
3_3_place_gp.json	2 KB
3_3_place_gp.log	24 KB
3_4_place_resized.json	2 KB
3_4_place_resized.log	17 KB
3_5_place_dp.json	3 KB
3_5_place_dp.log	10 KB
4_1_cts.json	3 KB
4_1_cts.log	13 KB
5_1_grt.json	5 KB
5_1_grt.log	113 KB
5_2_route.tmp.log	12 KB

Run logs (3)

asap7_cva6_h...26_004411.log	125 KB
asap7_cva6_h...25_235434.log	110 KB

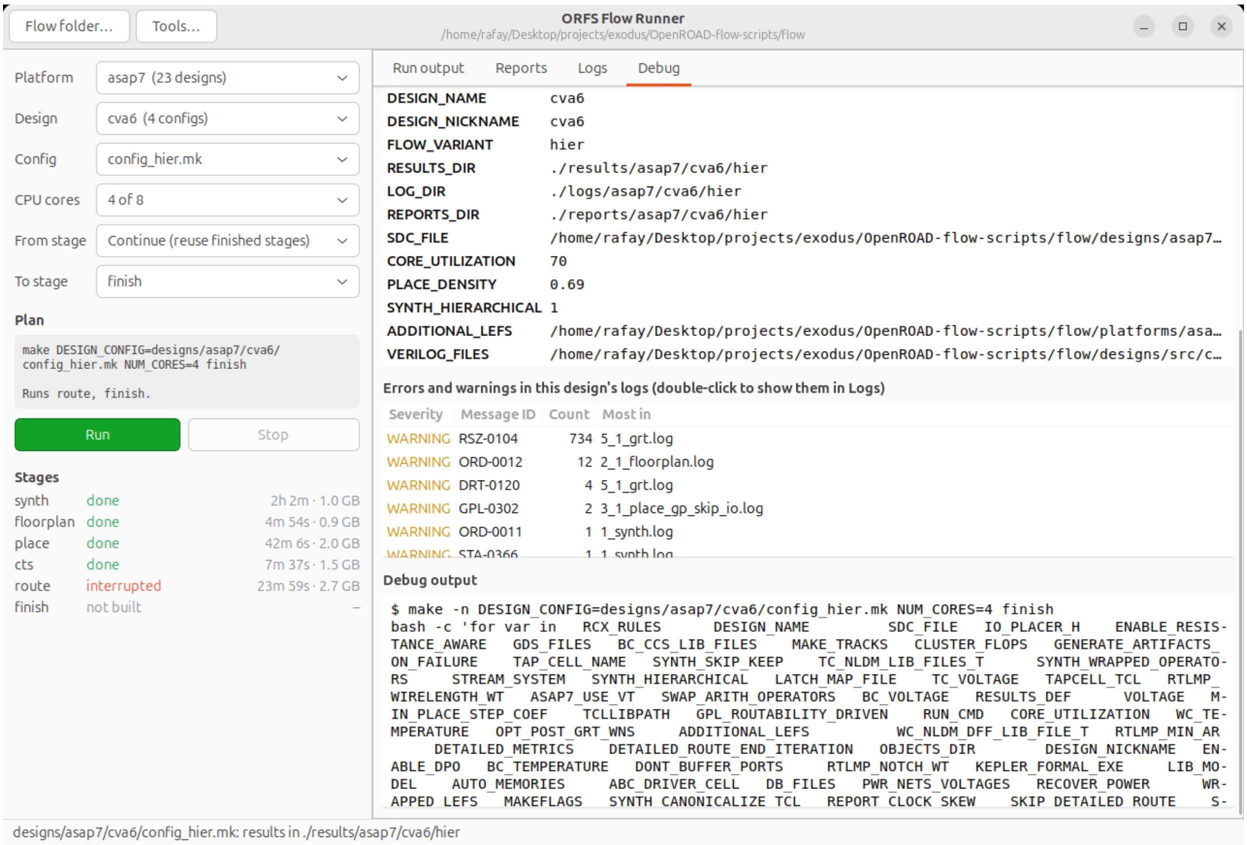
```
[INFO GRT-0020] Min routing layer: M2
[INFO GRT-0021] Max routing layer: M7
[INFO GRT-0022] Global adjustment: 0%
[INFO GRT-0023] Grid origin: (0, 0)
[INFO GRT-0088] Layer M1      Track-Pitch = 0.0360  line-2-Via P-
itch: 0.0360
[INFO GRT-0088] Layer M2      Track-Pitch = 0.0390  line-2-Via P-
itch: 0.0360
[INFO GRT-0088] Layer M3      Track-Pitch = 0.0360  line-2-Via P-
itch: 0.0360
[INFO GRT-0088] Layer M4      Track-Pitch = 0.0480  line-2-Via P-
itch: 0.0480
[INFO GRT-0088] Layer M5      Track-Pitch = 0.0480  line-2-Via P-
itch: 0.0480
[INFO GRT-0088] Layer M6      Track-Pitch = 0.0640  line-2-Via P-
itch: 0.0640
[INFO GRT-0088] Layer M7      Track-Pitch = 0.0640  line-2-Via P-
itch: 0.0640
[INFO GRT-0003] Macros: 8
[INFO GRT-0004] Blockages: 50622
[INFO GRT-0019] Found 1108 clock nets.
[INFO GRT-0001] Minimum degree: 2
[INFO GRT-0002] Maximum degree: 133
[INFO GRT-0053] Routing resources analysis:
[INFO GRT-0101] Running extra iterations to remove congestion.
[INFO GRT-0102] Start extra iteration 1/30
[INFO GRT-0197] Via related to pin nodes: 911305
[INFO GRT-0198] Via related Steiner nodes: 24634
[INFO GRT-0199] Via filling finished.
[INFO GRT-0111] Final number of vias: 1201807
[INFO GRT-0112] Final usage 3D: 4710362
[INFO GRT-0096] Final congestion report:
[INFO GRT-0018] Total wirelength: 852929 um
[INFO GRT-0014] Routed nets: 144880
[INFO GRT-0303] Global routing runtime = 00:05:16
[WARNING GRT-0246] No diode with LEF class CORE ANTENNACELL found
.
```

designs/asap7/cva6/config_hier.mk: results in ./results/asap7/cva6/hier

Figure 9. The global-route log filtered to GRT- messages; the run logs of the A/B runs are at the bottom of the list.

6.4 Debug

Debug has four parts. Actions: Dry run (make -n) shows what make would run for the plan; Open in OpenROAD GUI opens a built stage; Open results folder; Copy command copies the plan for a terminal. Environment (Figure 2). Resolved make variables: PLATFORM, FLOW_VARIANT, RESULTS_DIR, SDC_FILE, CORE_UTILIZATION and others as make print-<VAR> reports them. Errors and warnings: a count of every message ID in the design's logs, errors first; double-click one to open its log in Logs, filtered to that ID.



Platform: asap7 (23 designs)
Design: cva6 (4 configs)
Config: config_hier.mk
CPU cores: 4 of 8
From stage: Continue (reuse finished stages)
To stage: finish

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier.mk NUM_CORES=4 finish

Runs route, finish.
```

Stages

synth	done	2h 2m · 1.0 GB
floorplan	done	4m 54s · 0.9 GB
place	done	42m 6s · 2.0 GB
cts	done	7m 37s · 1.5 GB
route	interrupted	23m 59s · 2.7 GB
finish	not built	–

DESIGN_NAME	cva6
DESIGN_NICKNAME	cva6
FLOW_VARIANT	hier
RESULTS_DIR	./results/asap7/cva6/hier
LOG_DIR	./logs/asap7/cva6/hier
REPORTS_DIR	./reports/asap7/cva6/hier
SDC_FILE	/home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/designs/asap7…
CORE_UTILIZATION	70
PLACE_DENSITY	0.69
SYNTH_HIERARCHICAL	1
ADDITIONAL_LEFS	/home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/platforms/asa…
VERILOG_FILES	/home/rafay/Desktop/projects/exodus/OpenROAD-flow-scripts/flow/designs/src/c…

Errors and warnings in this design's logs (double-click to show them in Logs)

Severity	Message ID	Count	Most in
WARNING	RSZ-0104	734	5_1_grt.log
WARNING	ORD-0012	12	2_1_floorplan.log
WARNING	DRT-0120	4	5_1_grt.log
WARNING	GPL-0302	2	3_1_place_gp_skip_io.log
WARNING	ORD-0011	1	1_synth.log
WARNING	STA-0366	1	1_synth.log

Debug output

```
$ make -n DESIGN_CONFIG=designs/asap7/cva6/config_hier.mk NUM_CORES=4 finish
bash -c 'for var in    RCX_RULES       DESIGN_NAME         SDC_FILE    IO_PLACER_H    ENABLE_RESIS-
TANCE_AWARE   GDS_FILES   BC_CCS_LIB_FILES    MAKE_TRACKS   CLUSTER_FLOPS   GENERATE_ARTIFACTS_
ON_FAILURE    TAP_CELL_NAME   SYNTH_SKIP_KEEP    TC_NLDM_LIB_FILES_T       SYNTH_WRAPPED_OPERATO-
RS      STREAM_SYSTEM   SYNTH_HIERARCHICAL   LATCH_MAP_FILE    TC_VOLTAGE   TAPCELL_TCL    RTLMP_
WIRELENGTH_WT   ASAP7_USE_VT   SWAP_ARITH_OPERATORS   BC_VOLTAGE   RESULTS_DEF       VOLTAGE   M-
IN_PLACE_STEP_COEF    TCLLIBPATH   GPL_ROUTABILITY_DRIVEN    RUN_CMD   CORE_UTILIZATION   WC_TE-
MPERATURE   OPT_POST_GRT_WNS    ADDITIONAL_LEFS         WC_NLDM_DFF_LIB_FILE_T   RTLMP_MIN_AR
     DETAILED_METRICS    DETAILED_ROUTE_END_ITERATION   OBJECTS_DIR         DESIGN_NICKNAME   EN-
ABLE_DPO    BC_TEMPERATURE   DONT_BUFFER_PORTS     RTLMP_NOTCH_WT   KEPLER_FORMAL_EXE      LIB_MO-
DEL      AUTO_MEMORIES     ABC_DRIVER_CELL    DB_FILES    PWR_NETS_VOLTAGES    RECOVER_POWER      WR-
APPED_LEFS    MAKEFLAGS    SYNTH_CANONICALIZE_TCL    REPORT_CLOCK_SKEW    SKIP_DETAILED_ROUTE    S-
```

designs/asap7/cva6/config_hier.mk: results in ./results/asap7/cva6/hier

Figure 10. Debug scrolled down: the make variables, the warning IDs in the CVA6 logs (RSZ-0104 734 times, mostly in global routing) and a dry run.

7. Worked example: stock and parallel abc_new

7.1 What was compared

CVA6 synthesis spends most of its time in Yosys, and much of that in abc_new, which maps each of the 93 kept modules with ABC. The yosys/ submodule adds a parallel abc_new that maps independent modules at the same time. Runs from the app measure what that saves on a full ORFS synthesis.

	Stock	Parallel
Config	config_hier_stock.mk	config_hier_parallel.mk
Results folder	results/asap7/cva6/hier_stock	results/asap7/cva6/hier_parallel
Yosys (Tools...)	/usr/local/bin/yosys, 0.69+274	yosys/build/yosys, 0.69+280 (0.69+274 plus parallel abc_new)
Settings	NUM_CORES=8, From Continue, To synth	same

Both configs include config_hier.mk and change only FLOW_VARIANT, so the runs use identical settings and keep their results apart.

Method. The first attempt ran each build once, stock first, with a cold file cache: 47 min 55 s against 39 min 9 s. That is not a fair test, so it was repeated from scratch as four runs in the order stock, parallel, parallel, stock (26 October 2026, 13:17 to 16:22). Before every run the variant's results, logs, reports and objects were deleted, the CVA6 sources, the asap7 platform files, both Yosys builds and OpenROAD were read into memory, and the machine idled for 30 seconds. Every run was started from the app with Yosys chosen in Tools…. The machine is a VirtualBox VM, so other load on its host can affect it.

```
# orfs/flow/designs/asap7/cva6/config_hier_parallel.mk
include $(dir $(lastword $(MAKEFILE_LIST)))config_hier.mk
export FLOW_VARIANT = hier_parallel
```

7.2 How to repeat it

1. Build Yosys from the yosys/ submodule (README, Step 2).
2. In the app choose asap7 / cva6 / config_hier_stock.mk, To stage synth, and set Yosys to the stock build in Tools…. Press Run.
3. When it finishes, choose config_hier_parallel.mk, set Yosys to yosys/build/yosys in Tools..., and press Run.
4. Repeat in the order stock, parallel, parallel, stock, choosing Re-run from synth so each run starts from scratch, and compare the Stages times and the end of each variant's 1_2_yosys.log (Took … seconds: abc_new, the time per pass and the CPU time).
5. Compare the CPU time of the Yosys step between runs as well: it does the same work every time, so a run that used much more CPU time ran on a slower machine.

7.3 Results

Run	Build	Whole synthesis	Yosys step	abc_new	Yosys CPU time
1	stock	2,562 s (42:42)	2,510 s	1,427 s	3,054 s
2	parallel	2,255 s (37:35)	2,198 s	1,105 s	3,076 s
3	parallel	2,873 s (47:53)	2,800 s	1,711 s	3,890 s
4	stock	3,159 s (52:39)	3,084 s	1,453 s	3,664 s

Comparison	Whole synthesis	Yosys step	abc_new
Runs 1 and 2: machine at normal speed (CPU time about 3,060 s)	1.14×	1.14×	1.29×
Runs 4 and 3: machine slower (CPU time 3,660 to 3,890 s)	1.10×	1.10×	0.85×
Mean of the two runs of each build (stock 2,860 s, parallel 2,564 s)	1.12×	1.12×	1.02×

Each factor is the stock time divided by the parallel time, so above 1 means the parallel build was faster. In all four runs the netlists are identical apart from the Yosys version comment, with 115,493 instances and 18,297.4 µm². Peak memory of Yosys was 1,009 and 1,017 MB for stock and 1,047 and 1,045 MB for parallel. The most yosys-abc processes seen at once (5 s samples) were 1 for stock and 4 for parallel.

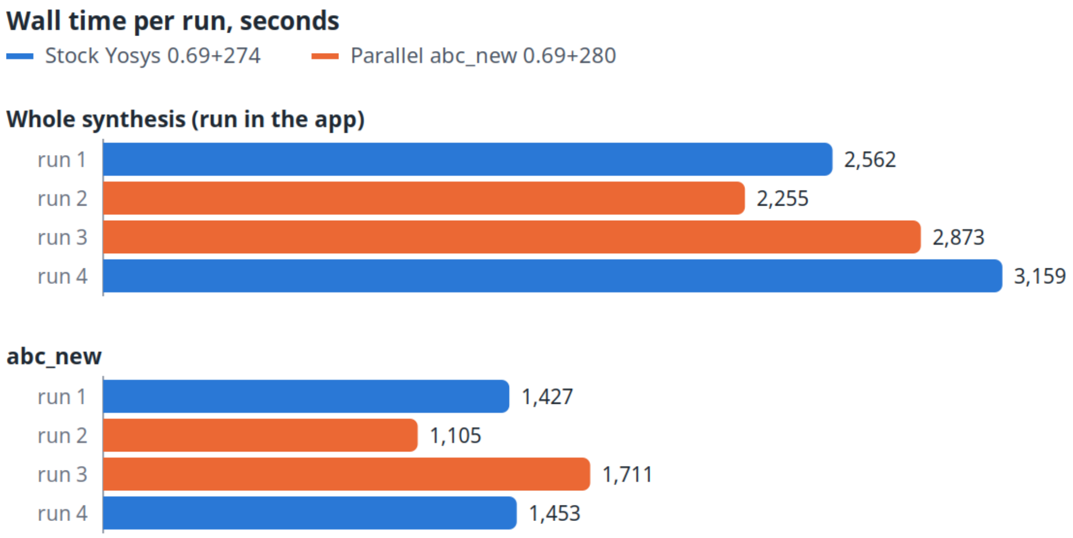


Figure 11. Wall time of each run: the whole synthesis and abc_new. Blue is stock, orange is parallel; bars are in run order.

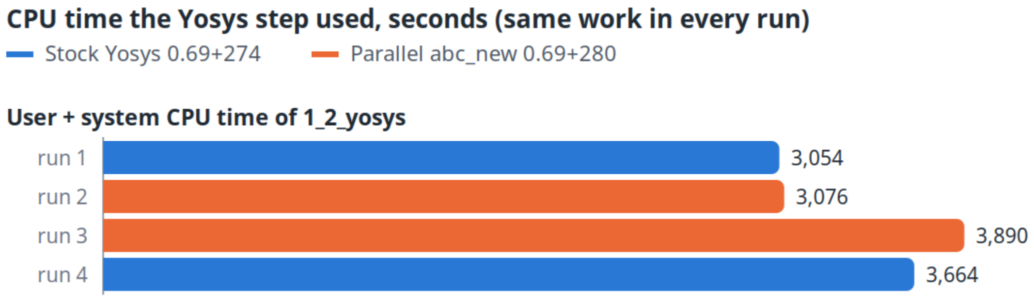


Figure 12. CPU time used by the Yosys step in each run. The work is the same every time, so runs 3 and 4 ran on a slower machine.

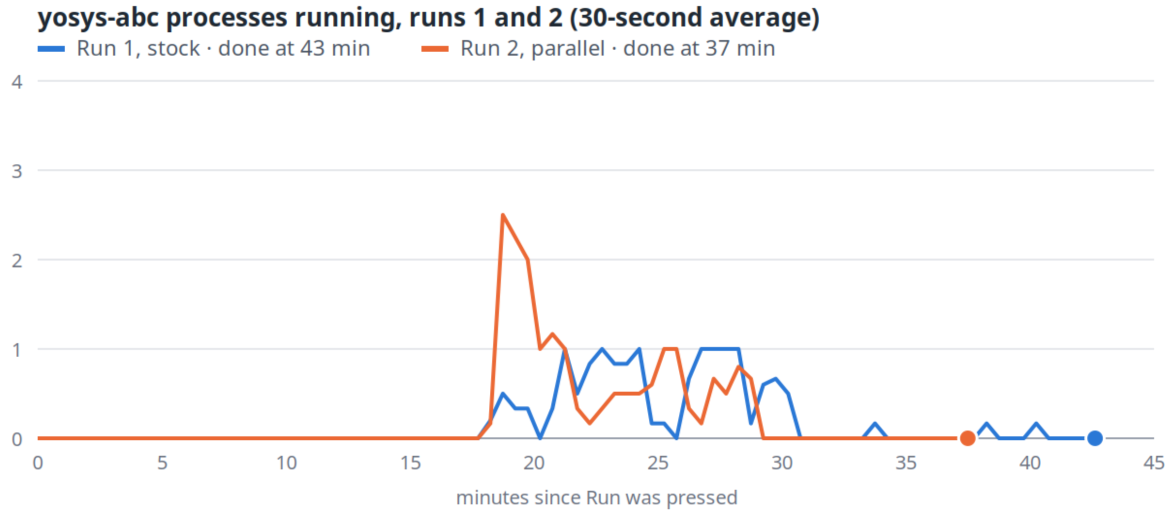


Figure 13. yosys-abc processes running in the cleanest pair, runs 1 and 2, averaged over 30 seconds. The parallel run reaches up to 4 processes; the stock run never has more than one.

7.4 What the numbers mean

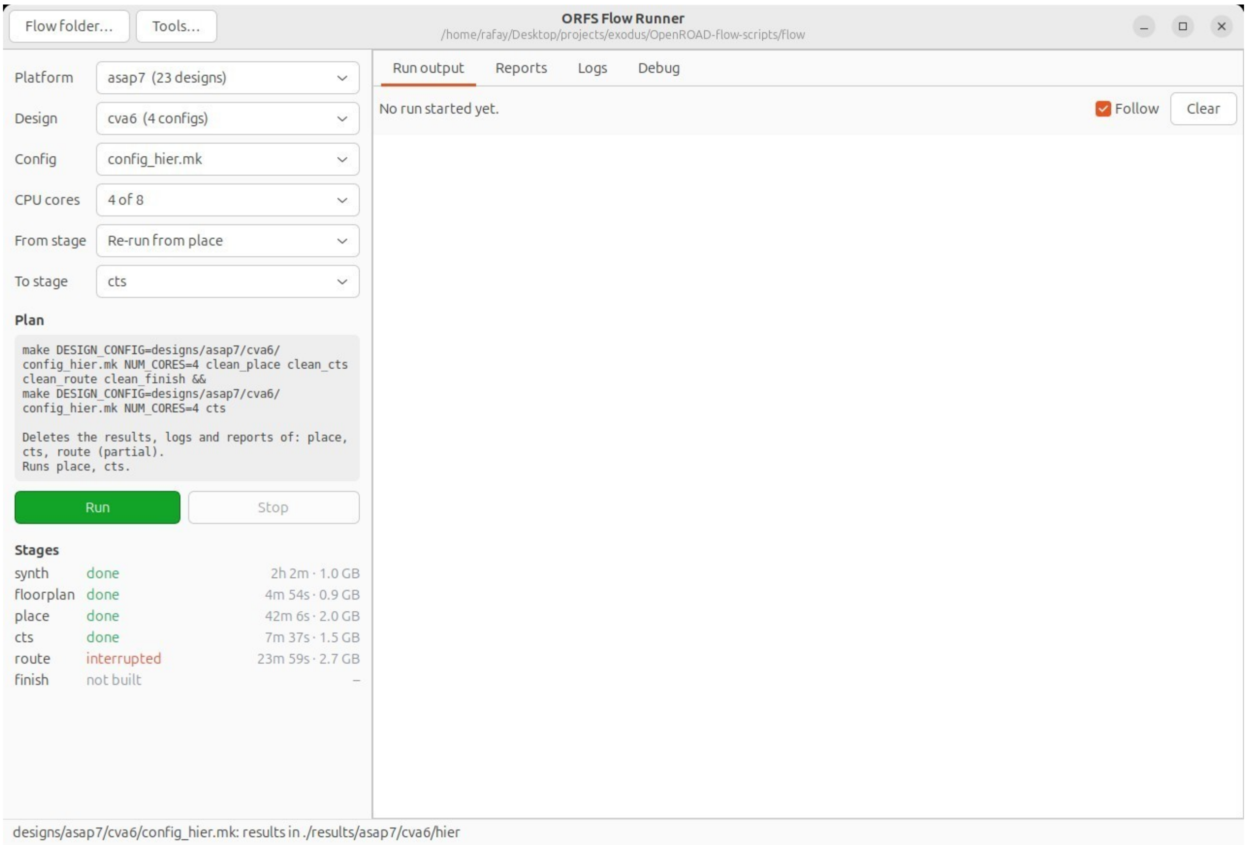
- **Correctness:** the parallel build maps exactly as stock does: the four netlists are identical, so any speedup is free of changes to the result.

- **Speedup:** in the cleanest pair, runs 1 and 2, abc_new was 1.29× faster and the whole synthesis 1.14×, in line with the 1.2× to 1.5× measured by replaying abc_new alone in the change report.
- **The machine was not steady:** runs 3 and 4 used 20 to 27% more CPU time for the same work (3,890 s and 3,664 s against about 3,060 s). The cause was not established; a shared VirtualBox host is the likely one. The slowdown lasted through both later runs, so the order stock, parallel, parallel, stock does not cancel it.
- **Noise is as large as the effect:** in the slowed pair the parallel abc_new took 1,711 s against 1,453 s for stock (0.85×), while the whole synthesis was still 1.10× faster. Averaged, abc_new is 1.02× and the whole synthesis 1.12×. Two runs per build cannot give a confidence interval; a fair summary is abc_new between 0.85× and 1.29× and the whole synthesis about 1.1× on this machine.
- **Parallelism stays low:** at most 4 ABC processes ran at once. Yosys still does serial work around each ABC call; in the parallel logs write_xaiger2 alone takes 352 and 464 s over 186 calls. That serial work caps the saving.
- **Next:** repeat on a quiet machine, ideally not a VM, with more runs per build, and use the CPU time of the Yosys step to reject runs that were slowed down. The earlier 47 min 55 s against 39 min 9 s (1.22×) is replaced by these results.

8. Other tasks

8.1 Re-run from a stage

Choose Re-run from <stage> in From stage. The plan lists the clean commands first, and Run asks for confirmation with the stages whose results, logs and reports will be deleted, partial ones included. Later stages are cleaned too, because they were built from the old results.



Platform: asap7 (23 designs)
Design: cva6 (4 configs)
Config: config_hier.mk
CPU cores: 4 of 8
From stage: Re-run from place
To stage: cts

Plan

```
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier.mk NUM_CORES=4 clean_place clean_cts
clean_route clean_finish &&
make DESIGN_CONFIG=designs/asap7/cva6/
config_hier.mk NUM_CORES=4 cts

Deletes the results, logs and reports of: place,
cts, route (partial).
Runs place, cts.
```

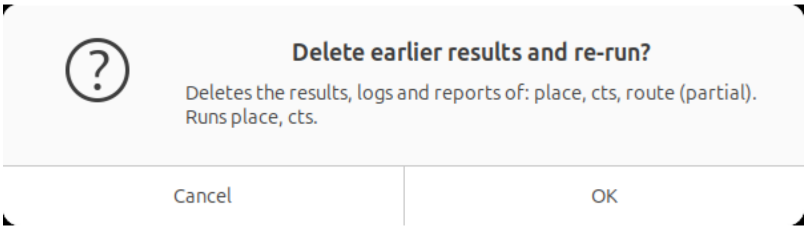
Stages

synth	done	2h 2m · 1.0 GB
floorplan	done	4m 54s · 0.9 GB
place	done	42m 6s · 2.0 GB
cts	done	7m 37s · 1.5 GB
route	interrupted	23m 59s · 2.7 GB
finish	not built	–

No run started yet.

designs/asap7/cva6/config_hier.mk: results in ./results/asap7/cva6/hier

Figure 14. Re-run from place to cts on the earlier CVA6 run: the plan cleans place, cts and route, then runs place and cts.



Delete earlier results and re-run?

Deletes the results, logs and reports of: place, cts, route (partial).
Runs place, cts.

Figure 15. The confirmation before anything is deleted.

8.2 Runs that must outlive the app

Closing the app asks whether to stop its run. For a run that must survive logging out, use Debug → Copy command and start it with systemd-run --user as in the README's Running unattended section. The app still shows that run's results and logs, but not its live status.

8.3 Open a stage in the OpenROAD GUI

Debug → Actions: pick a built stage in the drop-down and click Open in OpenROAD GUI. Its output goes to ~/.cache/orfs-flow-runner/openroad_gui.log.

9. Troubleshooting

Problem	**What to do**
No module named gi at start	sudo apt install python3-gi gir1.2-gtk-3.0, and use the system python3, not a venv

Problem	**What to do**
A run fails at once with error 127	OpenROAD or Yosys was not found: check Debug → Environment and choose the program with Tools…
The Platform menu is empty	the folder is not an ORFS flow/ folder: use Flow folder…
A stage shows interrupted	the step was killed, usually for lack of memory; look for oom-kill in journalctl, use fewer cores, then run again with Continue
read_slang missing (CVA6)	build Yosys with the slang frontend (README, Step 1 and 2)
Make variables show make error	the config does not parse; the message is in Debug output
The installed app does not appear in the menu	an older gui/install_launcher.sh entry hides it: run gui/install_launcher.sh --uninstall

Limits: one run at a time per window; only runs started from the app show as running; viewers show the last 4 MB of a log, and the full output is always in the run log.

10. Files

Path	**What it is**
gui/orfs_app.py	the app
gui/flow_gui.py	the CVA6 web console; the app reuses its metric definitions
gui/packaging/build_deb.sh	builds the .deb; orfs-flow-runner.desktop and .svg are its menu entry and icon
gui/install_launcher.sh	menu and Desktop entry for running from a checkout, without the package
orfs/flow/designs/asap7/cva6/config_hier_{stock,parallel}.mk	the A/B configs of section 7
~/.config/orfs-flow-runner/settings.json	flow folder, menu choices and tools
~/.local/share/orfs-flow-runner/run_logs/	one log per run: <platform>_<design>_<variant>_<from>-<to>_<date>_<time>.log
~/.cache/orfs-flow-runner/	OpenROAD GUI log; safe to delete
<flow>/results|logs|reports/<platform>/<design>/<variant>/	ORFS results (one .odb per stage, 6_final.gds), step logs with metrics JSON, and reports